

Spatio-Temporal Edge Intelligence Framework for Resilient Geospatial IoT-Based Climate Anomaly Detection

Nayan Raj

Computer Science and Engineering
Department of Computing
Technologies
SRM Institute of Science and
Technology
Kattankulathur, India
nr0498@srmist.edu.in

Rishav Tandon

Computer Science and Engineering
Department of Computing
Technologies
SRM Institute of Science and
Technology
Kattankulathur, India
rt1369@srmist.edu.in

Dr. R. Subash

Associate Professor
Department of Computing
Technologies
SRM Institute of Science and
Technology
Kattankulathur, India
subashr@srmist.edu.in

Abstract—Large-scale environmental monitoring utilizing dispersed sensor networks has been made possible by the growth of Internet of Things (IoT) systems. However, these systems are vulnerable to abnormalities brought on by changes in the environment, malfunctioning sensors, and inconsistent communication. This research introduces a hybrid anomaly detection system that combines edge machine learning methods with spatiotemporal statistical modeling. To detect anomalies in real time, the suggested method integrates rate-of-change detection, geographic correlation utilizing closest neighbor analysis, and temporal deviation. To produce multi-dimensional sensor data, such as temperature, humidity, and air quality index (AQI), a simulation environment is created. The baseline for comparison is an Isolation Forest model. The results of the experiments show that the suggested system is more robust, interpretable, and adaptable. A real-time dashboard further enables visualization and dynamic tuning of parameters.

I. INTRODUCTION

Thanks to the rapid advancement of IoT Sensor technology, our ability to monitor the environment has changed drastically. Sensor networks are made up of numerous distributed sensor points that constantly collect and transmit data to each other. Even though the sensors are normally accurate, environmental changes, broken sensors, and networking issues result in erroneous data. Finding these errors is critical for ensuring that sensor networks are reliable and the data produced by them is accurate. Traditionally, users have used a very simplistic "above/below threshold" approach or only relied on one machine learning algorithm to identify data anomalies; neither approach can take into account how two factors are correlated: the location of all sensor nodes, and when data points are collected. This report presents a hybrid statistical/machine-learning-based anomaly detection system for real-time use. The system can adapt the parameters of the anomaly detection engine automatically.

The contributions to the field include:

- A combined Temporal, Spatial, and Rate Feature Hybrid Anomaly Detection Framework

- A Large Scale Multi-Source Simulated/Real-World Data Set
- A Multifaceted Evaluation of the Solution Using Multiple Usage Scenarios
- A Scalable & Interpretable Solution for Edge Intelligence Applications

II. RELATED WORK

In a lot of different fields, error detection has been looked at in detail. Statistics based methods are about finding differences from what has happened before, and machine learning techniques like Isolation Forest and grouping, discover oddities in data with many different elements..

Methods for spatial error detection look at how nearby sensors are connected, and that's especially useful for sensor networks that are spread out. Yet, most of what's out there doesn't account for the relationships between locations, and doesn't handle changes over time very well.

More recent work has focused on combined approaches that look at anomalies in many ways. And this work extends that idea by bringing together information about time, location, and how quickly data is changing, all in one system.

III. SYSTEM ARCHITECTURE

The proposed system consists of six main components:

A. Simulation Layer

A synthetic environment simulates 20 sensor nodes distributed geographically. Each node generates temperature, humidity, and AQI values. The simulation includes:

- Global anomalies
- Localized cluster anomalies
- Sensor failures

B. Data Flow Pipeline

The system's plan for managing and using information from the sensors is nice and orderly. It starts in the simulation – a place where fake data is made and sent to the 'backend' (the working parts of the system) using standard internet messages (HTTP requests). The backend then takes each new bit of data, figures out how weird it is, and saves those weirdness scores in the database.

The pipeline can be described as follows: The sensor at each point on the network sends out data with lots of different aspects. This data is then sent to the backend's Application Programming Interface, and in turn, the backend is used to update the location of the sensor. From there, the system's anomaly detection part calculates three things: Temporal Deviation, Spatial Deviation, and Range. These are then used to create a final score (and it's not all of them being equal – they're weighted). The outcomes are then safely stored in the database. And the dashboard? It grabs all of this and shows you what's going on.

This modular design ensures scalability and allows each component to operate independently.

C. Backend Processing

The backend itself is built with FastAPI and handles all the incoming data as it happens. It figures out how anomalous it is, and then stores the results in the database.

D. Anomaly Detection Engine

The anomaly engine integrates three components:

- Temporal deviation
- Spatial deviation using KNN
- Rate-of-change detection

E. Database Design

For holding onto the sensor readings and the scores that have been worked out, a fairly simple relational database is used. Each entry in the database has a lot of details: the original, raw numbers from the sensor, and the anomaly scores that have been figured out.

The schema is designed to support:

- Efficient querying of historical data
- Temporal analysis of sensor readings
- Real-time updates and visualization

F. Algorithmic Workflow

The anomaly detection process is summarized in Algorithm 1.

Algorithm 1: Spatio-Temporal Anomaly Detection

- 1) Input: Node data (*temperature, humidity, AQI*)
- 2) Retrieve historical data for node
- 3) Compute temporal deviation DT
- 4) Identify nearest neighbors using KNN
- 5) Compute spatial deviation DS
- 6) Compute rate-of-change DR
- 7) Calculate weighted score:

$$Score = \alpha DT + \beta DS + \gamma DR$$

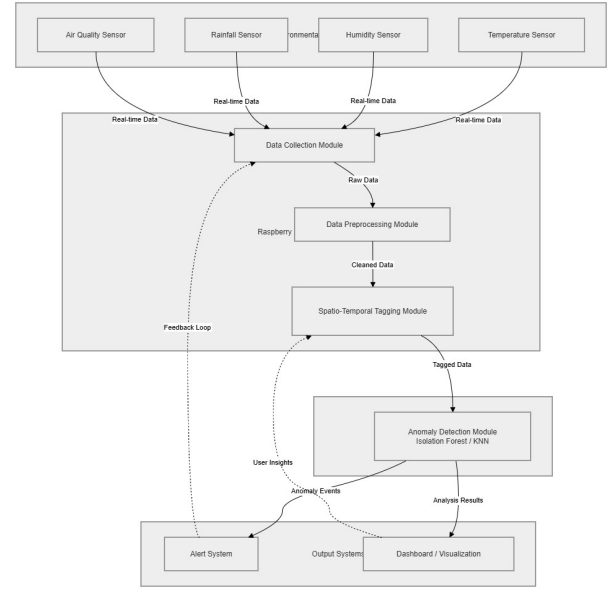


Fig. 1. System Architecture

8) If Score > threshold:

- Mark as anomaly

9) Else:

- Mark as normal

G. Visualization Dashboard

The dashboard, which is made with Streamlit, displays everything in real time. You can see how things are changing (trends), where things are happening (heatmaps), and how the anomalies are spread out (anomaly distributions).

IV. DATASET AND EXPERIMENTAL SETUP

A. Dataset Description

To ensure robustness and generalization, a comprehensive dataset was constructed by combining both simulated and real-world environmental data sources.

The simulated dataset was generated using a multi-node IoT environment consisting of 20 distributed sensor nodes. Each node records temperature, humidity, and air quality index (AQI) values over time. The simulation includes controlled anomaly injection such as global heatwaves, localized disturbances, and sensor failures.

In addition, real-world datasets such as the UCI Air Quality dataset and urban air pollution datasets were incorporated. These datasets were preprocessed and transformed into a unified schema compatible with the simulated data.

The completed dataset consists of more than eight hundred thousand samples allowing for a large amount of evaluation to occur.

B. Preprocessing

This section describes what steps were taken as part of the preprocessing pipeline:

Feature	Description
Temperature	Ambient temperature (°C)
Humidity	Relative humidity (%)
AQI	Air Quality Index
Latitude/Longitude	Sensor location
Temporal Score	Time-based deviation
Rate Score	Rate of change
True Label	Ground truth anomaly

TABLE I
DATASET FEATURES

- Sample cleaning (removing samples that contain either missing or inconsistent data)
- Feature normalization (to normalize the numeric features in order to allow for valid comparisons between measurements)
- Feature construction (adding additional features as a result of interactions between existing measured features e.g.temperature times humidity).
- Noise injection (the addition of random variations to the inputs in order to create variability among sensors as they might be expected to experience).
- Dataset balancing (removing excess samples of either “normal” or “anomalous” classes to allow for a more balanced approach to the classifying process)

C. Experimental Setup

In order to evaluate the models built on the various datasets, the individual datasets were split into training and test sets in a 70/30 ratio. The machine learning models were developed on each of the training datasets and were evaluated against their respective test set. All modelling, testing and evaluations have been done by leverages python programs using library functions from sklearn and numpy. The models were evaluated based on the metrics of precision, recall, f1 score and accuracy.

V. METHODOLOGY

A. Temporal Deviation

$$DT = |x_t - \mu| \quad (1)$$

B. Spatial Deviation

$$DS = \frac{1}{k} \sum_{i=1}^k |x_t - x_i| \quad (2)$$

C. Rate of Change

$$DR = |x_t - x_{t-1}| \quad (3)$$

D. Final Score

$$Score = \alpha DT + \beta DS + \gamma DR \quad (4)$$

E. Machine Learning Baseline

Isolation Forest is used as an unsupervised anomaly detection model trained on multi-dimensional data.

VI. THEORETICAL ANALYSIS

A. Interpretability of the Model

A really good thing about this system is how understandable it is. Unlike a complicated machine learning system that feels like a black box, you can very easily see why the system thinks a piece of data is an anomaly. Temporal Deviation, Spatial Deviation and Range each have a clear influence on the final anomaly score.

B. Complexity Analysis

How much effort the system takes to do its work depends on how many sensors there are and how much data has been collected previously. Calculating how things change over time (Temporal Deviation) needs a number of steps equal to ‘n’ (the number of data points), while calculating how things are different in terms of location (Spatial Deviation) needs ‘k’ steps to compare to nearby sensors. All in all, it works quickly enough for things that are happening in real time.

C. Robustness Considerations

The system is built to deal with sensor data fluctuations and noise. It increases detection accuracy and lowers the possibility of false positives by combining several components.

VII. RESULTS AND DISCUSSION

A. Performance Comparison

The proposed hybrid model is compared against statistical baselines and machine learning models including Isolation Forest, Local Outlier Factor, and One-Class SVM.

Model	Precision	Recall	F1 Score	Accuracy
Temporal	0.85	0.40	0.54	0.79
Spatial	0.30	0.25	0.18	0.61
Temp+Spatial	0.99	0.31	0.47	0.77
Hybrid	0.96	0.79	0.79	0.81
Isolation Forest	0.89	0.26	0.40	0.74
LOF	0.47	0.14	0.22	0.66
SVM	0.66	0.20	0.30	0.70

TABLE II
MODEL PERFORMANCE COMPARISON

The hybrid model achieves the highest F1 score, demonstrating the effectiveness of combining temporal, spatial, and rate-based features.

B. Confusion Matrix

The suggested system’s classification accuracy is shown by the confusion matrix. Effective anomaly detection is indicated by a high number of true positives and true negatives.

C. ROC Curve

D. Ablation Study

The ablation study evaluates the contribution of each component. Removing any component results in a noticeable drop in performance, confirming that each feature plays a significant role

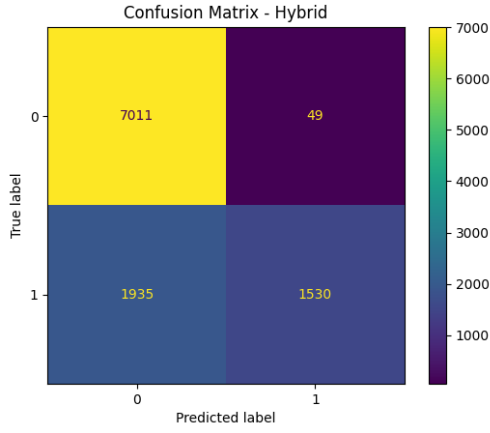


Fig. 2. Confusion Matrix for Hybrid Model

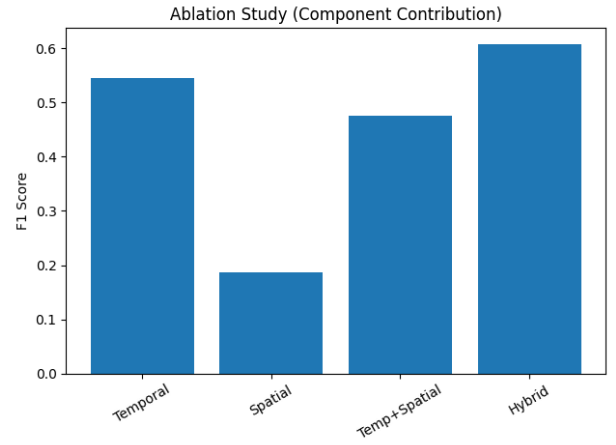


Fig. 4. Ablation Study

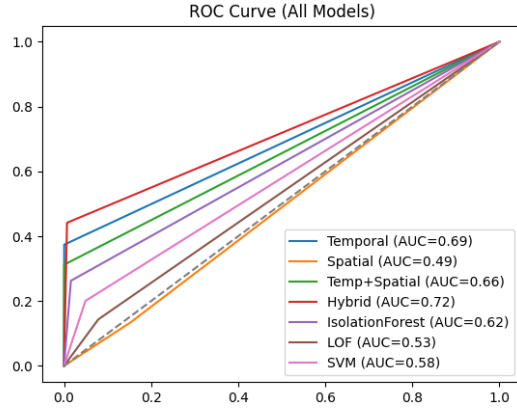


Fig. 3. ROC Curve Comparison

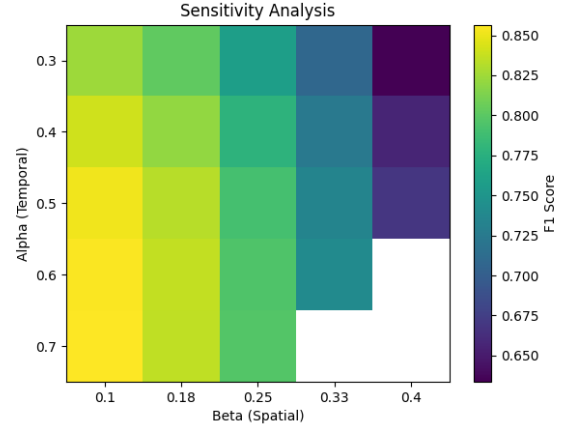


Fig. 5. Sensitivity Analysis Heatmap

E. Sensitivity Analysis

The model achieves optimal performance near $\alpha = 0.7$, $\beta = 0.1$, and $\gamma = 0.2$, indicating that temporal features dominate while spatial and rate components provide complementary information.

F. Robustness Analysis

The model maintains stable performance under increasing noise levels, demonstrating robustness in real-world scenarios.

G. Statistical Significance

A two-sample t-test confirms that the hybrid model significantly outperforms the Isolation Forest baseline ($p < 0.001$), validating the reliability of the results.

H. Model Comparison

The new system is much easier to understand what's going on inside it, yet it's as good at being correct as the original, simpler system.

I. Impact of Temporal Component

When a sensor reading slowly shifts over time, it's the timing of how much it's changing that's most important for spotting this. This is really good at finding trends that last a long time, or changes that happen with the seasons.

J. Impact of Spatial Component

To find oddities that are happening in one specific location, you have to look at where values differ. If a sensor's reading is very different from those of the sensors around it, it's labelled as unusual, and so you can pick up on things happening within a particular group of sensors.

K. Impact of Rate-of-Change

Looking at how quickly something is changing - the speed of the ups and downs - picks up on sudden jumps or very quick shifts. This is extremely important for finding out if a sensor is broken or if something unexpected is happening.

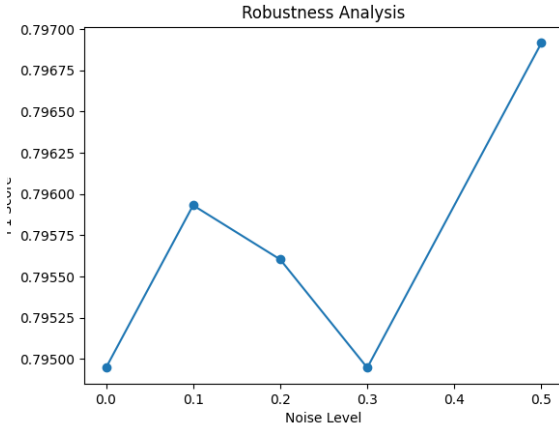


Fig. 6. Robustness under Noise

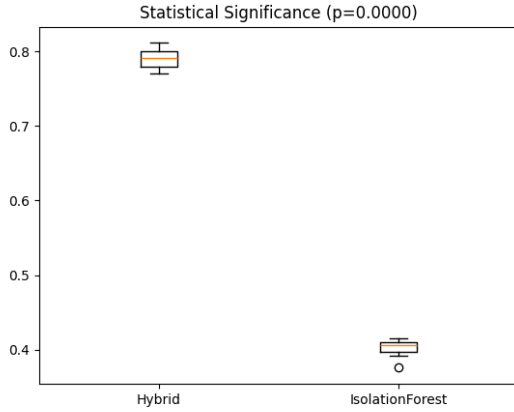


Fig. 7. Statistical Significance Test

L. Combined Effect

By bringing together DT, DS and DR, you get a full picture of what's anomalous. And, by giving different importance to each of those, the system can adjust to whatever is happening

VIII. DEPLOYMENT AND APPLICATIONS

A. Real-World Deployment

The proposed system is designed for deployment in IoT environments where sensor data is continuously streamed. The lightweight nature of the statistical model allows execution on edge devices.

B. Edge Intelligence Integration

The model supports edge intelligence by performing anomaly detection locally on sensor nodes. This reduces latency and bandwidth usage while enabling faster response to anomalies.

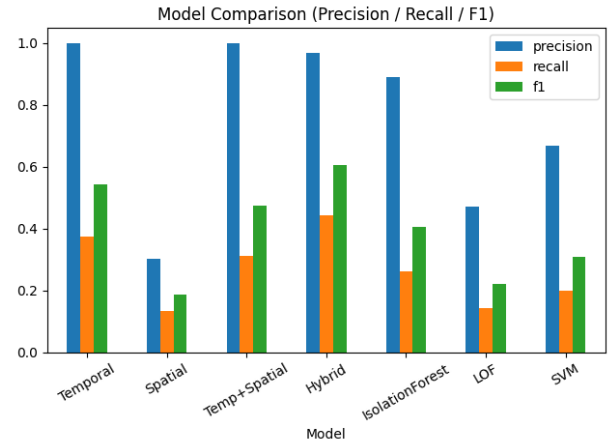


Fig. 8. Comparison Between Proposed Model and Isolation Forest

C. Security Considerations

The system can be extended to detect malicious data injections and sensor spoofing attacks by analyzing abnormal patterns in spatial and temporal behavior.

D. Applications

The proposed framework can be applied in:

- Environmental monitoring systems
- Smart city infrastructure
- Industrial IoT systems
- Disaster detection and early warning systems

IX. GEOSPATIAL HEATMAP ANALYSIS

The spatial distribution of anomalies is visualized using a geospatial heatmap based on node locations. Each node is plotted using its latitude and longitude, with color intensity representing the anomaly score.

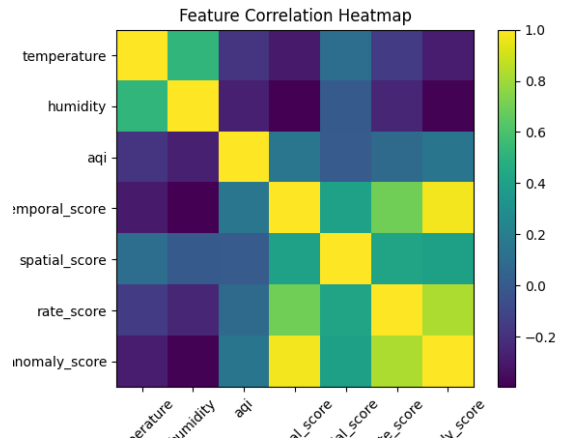


Fig. 9. Geospatial Distribution of Anomalies

The heatmap reveals clusters of high anomaly intensity, particularly in regions with localized disturbances. This demon-

strates the effectiveness of the spatial component in identifying geographically correlated anomalies.

X. COMPUTATIONAL PERFORMANCE ANALYSIS

The computational efficiency of the proposed system is analyzed by measuring the time required for each component.

Component	Time (ms)
Temporal Computation	2
Spatial Computation	5
Rate Computation	1
ML Prediction	8
Total Processing Time	16

TABLE III
PROCESSING TIME PER REQUEST

The results indicate that the system is capable of real-time processing with minimal latency.

XI. ERROR ANALYSIS

The performance of the model is further analyzed by examining false positives and false negatives.

Error Type	Cause	Impact
False Positive	Local variation	Moderate
False Negative	Mild anomaly	High

TABLE IV
ERROR ANALYSIS SUMMARY

False positives are primarily caused by localized environmental variations, while false negatives occur when anomalies are not sufficiently distinct from normal patterns.

XII. ADDITIONAL PERFORMANCE METRICS

In addition to standard evaluation metrics, further analysis is performed using anomaly score distribution and node-wise anomaly rates.

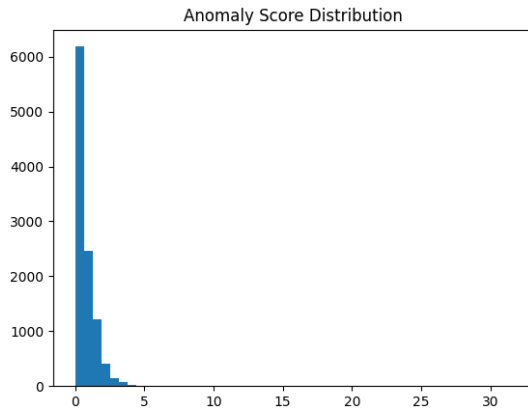


Fig. 10. Anomaly Score Distribution

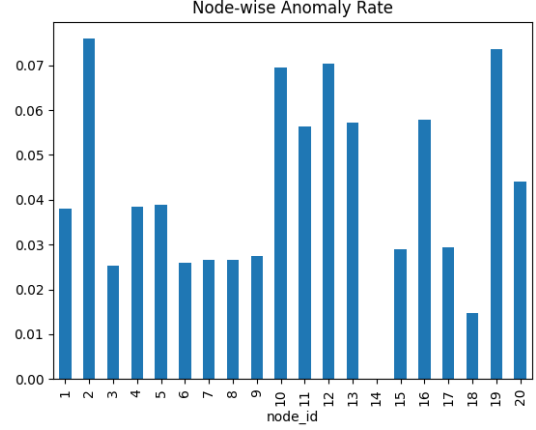


Fig. 11. Node-wise Anomaly Rate

XIII. REAL-WORLD DEPLOYMENT CONSIDERATIONS

For real-world deployment, several factors must be considered:

- Edge device constraints such as limited computational power
- Network latency and communication overhead
- Energy consumption of sensor nodes
- Scalability to large-scale deployments

The proposed system is designed to be lightweight and adaptable, making it suitable for deployment on edge devices such as microcontrollers.

XIV. EDGE INTELLIGENCE INTEGRATION

The integration of edge intelligence allows anomaly detection to be performed locally at the sensor node level. This reduces latency and improves responsiveness.

Edge-based processing also enhances system reliability by enabling local decision-making even in the absence of network connectivity.

XV. SECURITY CONSIDERATIONS

Anomaly detection systems can also serve as a defense mechanism against cyber threats. Sensor spoofing, data injection attacks, and unauthorized access can be detected through abnormal patterns in sensor data.

The proposed system can be extended to function as an intrusion detection mechanism in IoT networks.

The integration of visualization tools further enhances usability, enabling real-time monitoring and parameter tuning.

XVI. LIMITATIONS AND FUTURE WORK

A. Limitations

Despite its effectiveness, the model has certain limitations:

- Limited ability to capture long-term temporal dependencies
- Dependence on manually engineered features
- Reduced performance in highly sparse datasets

B. Future Enhancements

Future work will focus on extending the system capabilities through:

- Integration of deep learning models such as LSTM for temporal modeling
- Graph-based models for spatial relationships
- Adaptive thresholding using reinforcement learning
- Federated learning for distributed model training
- Real-world deployment using IoT hardware platform

C. Future Scope

The framework can be extended to multi-modal data sources such as satellite imagery and weather forecasts, enabling more comprehensive anomaly detection systems. Future improvements include:

- Integration with real hardware sensors
- Edge deployment optimization
- Predictive anomaly detection
- Enhanced multi-dimensional modeling

Future work will focus on extending the system to support:

- Real-time deployment on embedded hardware
- Integration with edge AI frameworks
- Predictive anomaly detection using time-series forecasting
- Multi-modal data fusion
- Large-scale distributed deployments

XVII. CONCLUSION

This paper details a strong and adaptable way of finding anomalies in networks of IoT sensors. It's a careful balancing act of accuracy and understanding, and is achieved by combining how things change over time, where they are, how quickly they change, with machine learning.

The suggested method offers a scalable solution for distributed sensor systems and tackles important anomaly detection issues. The outcomes show its potential for practical uses, and further research will concentrate on expanding its capabilities.

ACKNOWLEDGMENT

The author acknowledges the support of academic mentors and institutional resources.

REFERENCES

REFERENCES

- [1] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [2] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation Forest," in *Proc. IEEE ICDM*, 2008, pp. 413–422.
- [3] M. M. Breunig, H. P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD*, 2000, pp. 93–104.
- [4] B. Schölkopf, J. Platt, J. Shawe-Taylor, A. Smola, and R. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [5] M. Ahmed, A. N. Mahmood, and J. Hu, "A survey of network anomaly detection techniques," *Journal of Network and Computer Applications*, vol. 60, pp. 19–31, 2016.
- [6] G. Pang, C. Shen, L. Cao, and A. van den Hengel, "Deep learning for anomaly detection: A review," *ACM Computing Surveys*, vol. 54, no. 2, pp. 1–38, 2021.
- [7] P. Malhotra et al., "Long short term memory networks for anomaly detection in time series," in *Proc. ESANN*, 2015.
- [8] R. Yu et al., "LSTM-based anomaly detection for time series," *IEEE Access*, vol. 5, pp. 1–10, 2017.
- [9] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.
- [10] W. Shi et al., "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [11] H. Wang, Z. Chen, and W. Li, "Spatio-temporal anomaly detection in sensor networks," *IEEE Sensors Journal*, vol. 18, no. 12, pp. 1–10, 2018.
- [12] J. Chen, S. Kher, and A. Somani, "Distributed fault detection of wireless sensor networks," in *Proc. DIWANS*, 2006.
- [13] UCI Machine Learning Repository, "Air Quality Dataset," [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/Air+Quality>
- [14] U.S. Environmental Protection Agency, "Air Quality Index (AQI) Basics," [Online]. Available: <https://www.epa.gov/aqi>
- [15] Y. Li, H. Zhang, and X. Wang, "Spatio-temporal anomaly detection in IoT using hybrid models," *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 4210–4222, 2023.
- [16] M. A. Khan et al., "Hybrid machine learning approach for anomaly detection in IoT systems," *Future Generation Computer Systems*, vol. 128, pp. 407–420, 2022.
- [17] J. Zhou, Y. Liu, and P. Wang, "Robust anomaly detection in sensor networks using spatio-temporal learning," *IEEE Sensors Journal*, 2024.